Chapter 34 - IE Script

IE Script is the batch-control partner to IE Mon. IE Mon is typed one command at a time; IE Script stores a sequence of commands and runs them as one job. Use it for repeatable setup, frame waits, VideoChip checks, breakpoint sessions, and fault capture.

IE Script is not the normal way to write CPU programmes. BASIC and IE Mon remain the native programming route. IE Script automates the machine while a programme is running.

34.1 Running a Script

A stored script uses the `.IES` suffix. From BASIC, run it with:

```
RUN "FIRST.IES"
```

From IE Mon, run the same stored script with:

```
(ie64)> script FIRST.IES
```

When a script finishes, BASIC or IE Mon regains control. If it raises an error, the message is printed and control returns to the surface that launched it.

34.2 Language Shape

An IE Script is a sequence of statements. It supports integers, booleans, strings, arithmetic, comparison, tables, functions, `if`/`then`/`elseif`/`else`/`end`, `while`, and `for`.

Comments begin with `--`:

```
-- Wait for two video frames, then print a line.
sys.wait_frames(2)
sys.print("ready")
```

Function names are lower case. Module names are lower case. Strings may use single or double quotes.

34.3 System Module

`sys` handles time, text output, script exit, and storage helpers.

Function	Purpose
`sys.wait_frames(n)`	Yield until `n` video frames have passed.
`sys.wait_until(fn, max)`	Test `fn()` immediately, then after each frame until it is truthy or the optional frame budget expires.
`sys.wait_ms(ms)`	Yield until `ms` milliseconds have passed.
`sys.print(text)`	Print text to the terminal.

Function	Purpose
`sys.log(text)`	Write a diagnostic line.
`sys.time_ms()`	Current monotonic time in milliseconds.
`sys.frame_count()`	Number of completed frames.
`sys.frame_time()`	Most recent frame time.
`sys.fps()`	Current frame-rate estimate.
`sys.perf_report()`	Return the subsystem performance report string.
`sys.perf_reset()`	Clear subsystem performance counters.
`sys.quit()`	Stop the script and return to the caller.
`sys.exit(code)`	Stop Intuition Engine with status `code`.
`sys.mkdir(name)`	Create a directory in approved script storage.
`sys.read_file(name)`	Return stored bytes as a string.
`sys.write_file(name, data)`	Write bytes to approved script storage.
`sys.copy_file(from, to)`	Copy stored bytes.
`sys.capture_output(name)`	Start terminal output capture.
`sys.capture_output_off()`	Stop terminal output capture.

Long loops should call `sys.wait_frames` or `sys.wait_ms`; this keeps cancellation responsive.

`sys.wait_until` returns true when its predicate becomes truthy and false when its frame budget expires. Omit the budget, or pass `nil`, to wait indefinitely. An explicit budget must be numeric; it is truncated towards zero to form the frame count, and that resulting count must be non-negative. The count measures frame notifications. Zero performs only the immediate test. An error raised by the predicate remains a script error.

This script waits for three completed frames without writing its own polling loop:

```
local target = sys.frame_count() + 3
local reached = sys.wait_until(function()
  return sys.frame_count() >= target
end, 3)
sys.print("REACHED " .. tostring(reached))
```

The predicate is tested once before the first frame wait, then once after each of the three permitted frames. The expected line is `REACHED true`. Changing the budget to `0` prints `REACHED false` unless the target was already reached during the immediate test.

`sys.perf_reset()` and `sys.perf_report()` are for repeatable measurement scripts. The report is empty when performance accounting is off, or when no instrumented subsystem path ran during the measured span. When it is non-empty, each line gives a bucket name, total time, operation count, and average time per operation. The current subsystem buckets include video frame work, audio pulls, slow bus `Read32` and `Write32`, and Voodoo swap stages.

34.4 CPU Module

`cpu` controls the selected CPU:

Function	Purpose
`cpu.load(name)`	Load and start a stored CPU programme.
`cpu.load_stopped(name)`	Load a stored IE64, IE32, or Z80 programme but leave it stopped.
`cpu.reset()`	Reset the selected CPU.
`cpu.freeze()`	Stop the selected CPU for safe raw RAM access.
`cpu.resume()`	Resume after `cpu.freeze()`.
`cpu.start()`	Start execution.
`cpu.stop()`	Stop execution.
`cpu.is_running()`	Return true if the selected CPU is running.
`cpu.mode()`	Return the selected CPU name.
`cpu.execution_mode()`	Return the current execution mode name.
`cpu.jit_stats()`	Return execution statistics for the selected CPU.

Raw RAM access through `mem` requires the CPU to be frozen. MMIO access is allowed while the CPU is running.

34.4.1 Inspecting CPU Execution

`cpu.jit_stats()` returns a table owned by the selected CPU. If that CPU is unavailable, the table is empty. Where a `backend` field is present, its value is `native`, `wasm`, or `none`.

Each processor has a different execution engine, so each returns a different field set:

CPU	Returned fields
IE64	`backend`, `instruction_count`, `native_entries`, `native_retired`, `compiled_blocks`, `compiled_regions`, `region_candidates`, `region_rejections`, `fallback_instructions`, `helper_exits`, `helper_resumes`, `helper_resume_cancellations`, `io_bails`, `invalidations`, `cache_hits`, `cache_misses`, `spills`, `fpu_spills`, `direct_ram_proofs`, `inlined_calls`
IE32	`backend`, `instruction_count`, `native_entries`, `compiled_blocks`, `compiled_regions`, `hot_recompilations`, `retired_instructions`, `direct_instructions`, `helper_instructions`, `helper_exits`, `helper_resumes`, `chains`, `chain_budget_exits`, `deoptimizations`, `helper_deopts`, `source_stamp_deopts`, `code_cache_resets`, `invalidations`, `invalidated_blocks`, `cache_hits`, `return_cache_hits`, `mmio_poll_iterations`, `mmio_store_helpers`, `resident_spills_saved`, `counted_loops`, `profitability_fallbacks`
M68K	`instruction_count`, `native_blocks`, `native_retired`, `native_chain_instructions`, `native_no_chain_returns`, `native_helper_exits`, `native_exception_exits`, `native_invalidation_exits`, `native_mmio_guard_exits`, `unsupported_one_exits`, `compile_failure_exits`, `transcendental_bursts`, `warmup_instructions`, `region_promotions`, `last_native_pc`, `fallback_instructions`, `bailouts`, `last_fallback_pc`, `last_fallback_opcode`, `fallback_opcodes`, `native_pcs`, `native_invalidation_pcs`, `native_pc_ring`, `compile_failures`
6502	`instruction_count`, `tier1_blocks`, `native_entries`, `bailouts`, `invalidations`, `chain_exits`
Z80	`backend`, `instruction_count`, `native_entries`, `helper_exits`, `bailouts`, `invalidations`, `chain_exits`, `region_promotions`
x86	`backend`, `instruction_count`, `native_entries`, `native_retired`, `compiled_blocks`, `compiled_regions`, `region_candidates`, `fallback_instructions`, `helper_exits`, `io_bails`, `invalidations`, `invalidated_blocks`, `chain_exits`, `cache_hits`, `cache_misses`, `code_cache_resets`

For IE64, `instruction_count` is the processor's total retired-instruction count. For x86, it is the work accounted by the selected execution path and equals `native_retired` plus `fallback_instructions`. `native_entries` counts entries into generated code. `native_retired` counts instructions completed there. `fallback_instructions` counts instructions completed by the fallback path while accelerated execution remains active. These distinctions matter because `cpu.execution_mode()` reports the selected execution mode, not proof that a particular programme reached generated code.

The compilation fields count installed blocks, installed regions, and region attempts. Cache fields count lookup outcomes. Invalidation fields show that programme writes removed compiled code. Helper and I/O fields show work handed back for individual operations. On IE64, the resume fields describe continuations after a helper; the spill, direct-RAM, and inlined-call fields describe cumulative properties of installed compilation units.

IE64 and x86 statistics belong to one CPU instance. Resetting that CPU or loading another programme clears its table without changing another instance's counters.

This script prints a small summary without assuming that every processor uses the same field names:

```
local s = cpu.jit_stats()
local entered = s.native_entries or s.native_blocks or 0
local retired = s.native_retired or s.retired_instructions or 0
local fallback = s.fallback_instructions or 0

sys.print("CPU " .. cpu.mode())
sys.print("MODE " .. cpu.execution_mode())
sys.print("NATIVE ENTRIES " .. tostring(entered))
sys.print("NATIVE RETIRED " .. tostring(retired))
sys.print("FALLBACK " .. tostring(fallback))
```

Run it after the programme has done representative work. The first two lines identify the selected CPU and execution mode. The remaining lines show whether generated code was entered, how much work it retired, and how much work fell back where that counter exists. A zero can be meaningful: a programme may not yet be hot, a backend may be unavailable, or that CPU may report the same event under a different field.

`cpu.load_stopped` is supported for IE64, IE32, and Z80. It reads the stored image, resets the selected CPU, loads the bytes, and leaves the CPU stopped so that registers, memory, and execution mode can be set before `cpu.start()`. IE64 and Z80 reject an oversized image before disturbing the running programme. A Z80 image starts at the selected Z80 load address and must fit within the banked visible range. Invalid storage names, read failures, oversized images, unavailable CPUs, and other CPU selections raise a script error.

With Z80 selected, this script creates a two-byte programme, loads it while stopped, sets the accumulator, runs it, and inspects the result:

```
-- INC A; HALT
sys.write_file('PROBE.IE80', string.char(0x3C, 0x76))
cpu.load_stopped('PROBE.IE80')
dbg.set_reg('A', 41)
sys.print('STOPPED ' .. tostring(not cpu.is_running()))

cpu.start()
sys.wait_ms(10)
cpu.stop()
sys.print('A ' .. dbg.get_reg('A'))
```

The first printed line is `STOPPED true`, proving that loading did not start the CPU. The programme increments `A` once and halts. After the script stops the CPU, the second line is `A 42`. Try changing the value passed to `dbg.set_reg` from `41` to `9`; the final line becomes `A 10`.

34.5 Memory Module

`mem` reads and writes the bus:

Function	Purpose
`mem.read8(addr)`	Read one byte.
`mem.read16(addr)`	Read one little-endian word.
`mem.read32(addr)`	Read one little-endian long.
`mem.write8(addr, value)`	Write one byte.
`mem.write16(addr, value)`	Write one word.
`mem.write32(addr, value)`	Write one long.
`mem.read_block(addr, count)`	Return `count` bytes as a string.
`mem.write_block(addr, data)`	Write the bytes of `data`.
`mem.fill(addr, count, byte)`	Fill a range with one byte.

If a raw RAM read or write is attempted while the CPU is not frozen, the script raises an error. Use `cpu.freeze()` before the access and `cpu.resume()` afterwards.

34.6 Coprocessor Module

`coproc` starts workers and exchanges request strings through the same mailbox used by BASIC and raw MMIO:

Function	Purpose
`coproc.start(cpu_type, filename [, instance])`	Start a service; instance defaults to `0`.
`coproc.stop(cpu_type [, instance])`	Stop a service; instance defaults to `0`.
`coproc.enqueue(cpu_type, op, request [, instance])`	Submit request bytes and return a ticket.
`coproc.poll(ticket)`	Return the ticket status name.
`coproc.wait(ticket, timeout_ms)`	Wait and return status plus response bytes.
`coproc.response(ticket)`	Return available response bytes.
`coproc.stats()`	Return operation, byte, overhead, and completion counters.
`coproc.workers()`	Return all live worker type and instance pairs.

CPU type names are `ie32`, `ie64`, `6502`, `m68k`, `z80`, and `x86`. M68K, x86, and IE64 accept instances `0` and `1`; the other types accept only `0`. Invalid types, invalid instances, and failed commands raise a script error. `coproc.workers()` reads the per-instance state mask without changing the current raw MMIO selectors.

34.7 Terminal Module

`term` drives keyboard, mouse, and terminal text:

Function	Purpose
`term.type(text)`	Type text into the terminal.
`term.type_line(text)`	Type text followed by Return.
`term.read()`	Read pending terminal output.
`term.clear()`	Clear terminal output.
`term.echo(on)`	Enable or disable local echo.
`term.wait_output(text, timeout)`	Wait for text to appear.
`term.mouse_move(x, y)`	Move the mouse pointer.
`term.mouse_delta(dx, dy)`	Move the mouse pointer by a delta.
`term.mouse_click(button)`	Press and release a mouse button.
`term.mouse_release(button)`	Release a mouse button.
`term.scancode(code)`	Inject a keyboard scancode.
`term.key_press(code)`	Press and release a key.

34.8 Audio Module

`audio` controls the mixer and supported playback engines:

Function group	Purpose
`audio.start()`, `audio.stop()`, `audio.reset()`	Control audio output.
`audio.freeze()`, `audio.resume()`	Pause and resume audio processing.
`audio.write_reg(addr, value)`	Write an audio MMIO register.
`audio.write_regs(pairs)`	Apply an ordered list of `{addr, value}` 32-bit bus writes in one call.
`audio.set_master_gain_db(v)`	Set master gain in dB.
`audio.get_master_gain_db()`	Read master gain in dB.
`audio.set_master_auto_level_enabled(on)`	Enable automatic master levelling.
`audio.set_master_compressor_enabled(on)`	Enable master compression.
`audio.psg_load/play/stop/is_playing/metadata`	PSG playback helpers.
`audio.sid_load/play/stop/is_playing/metadata`	SID playback helpers.
`audio.ted_load/play/stop/is_playing`	TED playback helpers.
`audio.pokey_load/play/stop/is_playing`	POKEY playback helpers.
`audio.ahx_load/play/stop/is_playing`	AHX playback helpers.
`audio.midi_load/play/stop/pause/resume/set_volume/is_playing/metadata`	MIDI/MUS playback helpers.

`audio.write_regs` converts each address and value to an unsigned 32-bit quantity and performs the writes in list order, exactly as sequential `audio.write_reg` calls would. Every entry must be a table with at least two elements. A malformed entry raises an argument error; valid entries before it have already been written and are not undone.

Live MIDI has no separate high-level script helper. A script that deliberately wants to drive it can use `audio.write_reg(0xF0BF4, byte)` for data bytes and `audio.write_reg(0xF0BF6, 1)` for reset, then use `dbg.io("midilive")` to inspect the port.

34.9 Video Module

`video` controls display chips, blitter operations, and frame inspection:

Function group	Purpose
`video.write_reg(addr, value)`, `video.read_reg(addr)`	Raw video MMIO.
`video.get_dimensions()`, `video.is_enabled()`	Current VideoChip state.
`video.vga_enable(on)`, `video.vga_set_mode(mode)`	VGA control.
`video.vga_set_palette(i, r, g, b)`, `video.vga_get_palette(i)`	Write or read one VGA palette entry.
`video.ula_enable(on)`, `video.ula_border(n)`	ULA control.
`video.antic_enable(on)`, `video.antic_dlist(addr)`	ANTIC control.
`video.gtia_color(i, value)`	GTIA colour register write.
`video.ted_enable(on)`, `video.ted_mode(a, b)`	TED control.
`video.voodoo_enable(on)`, `video.voodoo_draw()`	Voodoo control.
`video.copper_enable(on)`, `video.copper_set_program(addr)`	Copper control.
`video.blit_copy(...)`, `video.blit_fill(...)`, `video.blit_line(...)`	Blitter commands.
`video.blit_wait()`	Wait until the blitter is idle.
`video.get_pixel(x, y)`	Return one composited RGBA pixel.
`video.get_region(x, y, w, h)`	Return a rectangle of composited RGBA bytes.
`video.frame_hash()`	Hash the current frame.
`video.wait_pixel(...)`	Wait for one pixel to match.
`video.wait_stable(frames, timeout)`	Wait for a stable frame hash.
`video.wait_condition(fn, timeout)`	Wait until callback `fn` returns true.
`video.is_crt_enabled()`	Return true for flat or curved CRT presentation.
`video.set_crt_enabled(on)`	Select flat presentation when true, or turn CRT presentation off.
`video.toggle_crt()`	Toggle between flat and off; return the new enabled state.
`video.get_crt_mode()`	Return `"flat"`, `"curved"`, or `"off"`.
`video.set_crt_mode(mode)`	Select `"flat"`, `"curved"`, or `"off"`.

Function group	Purpose
`video.cycle_crt_mode()`	Advance flat, curved, off, flat; return the new mode.
`video.get_scale_mode()`	Return `"fit"` or `"stretch"`.
`video.set_scale_mode(mode)`	Select `"fit"` or `"stretch"`.

34.9.1 VGA palette entries

`video.vga_set_palette(i, r, g, b)` masks `i` to eight bits and writes one contiguous three-byte RGB entry. Each component is stored as its low six bits. `video.vga_get_palette(i)` applies the same index mask and returns the three stored values, each in the range `0` to `63`.

This example deliberately uses index `257` and component values wider than six bits so that both rules are visible:

```
video.vga_set_palette(257, 255, 64, 130)
r, g, b = video.vga_get_palette(1)
sys.print('VGA ' .. r .. ' ' .. g .. ' ' .. b)
```

The index wraps to entry `1`. The stored components are `63`, `0`, and `2`, so the script prints:

```
VGA 63 0 2
```

The call changes palette RAM only. Select a VGA indexed-colour mode and draw with palette index `1` when you also want to see the colour.

34.9.2 Presentation scale

Presentation scale controls how a source rectangle is placed in the final output. `stretch` is the default. It fills the output even when that changes the source aspect ratio. `fit` preserves the aspect ratio and centres the source in the largest matching rectangle. A source which already matches the output aspect ratio looks the same in either mode.

The calls report and change presentation state only. They do not write video MMIO, alter source framebuffer bytes, or inject a key into the running programme.

This example saves the current mode, selects `fit`, checks it, then restores the saved mode:

```
old_scale = video.get_scale_mode()
video.set_scale_mode('fit')
sys.print('SCALE ' .. video.get_scale_mode())

video.set_scale_mode(old_scale)
sys.print('RESTORED ' .. video.get_scale_mode())
```

The first line printed is `SCALE fit`. On an unchanged startup the second is `RESTORED stretch`, because `stretch` is the default. If a script inherited a different mode, the second line names that restored mode instead.

`video.get_scale_mode()` raises a script error when the selected output has no compositor. `video.set_scale_mode()` raises the same error, and also rejects every name except `fit` and `stretch`.

34.9.3 CRT presentation and two-stage capture

CRT mode changes the final presentation of the completed picture. It does not change video registers, framebuffer bytes, palette values, or the composited pixel returned by `video.get_pixel`. The explicit mode calls use the same three-state

order as F7, but they do not inject an F7 key into the running programme.

The older boolean calls remain useful for scripts that need only on or off. Enabling selects flat mode. `video.toggle_crt()` changes flat or curved to off, and changes off to flat. It never selects curved mode.

This example captures the picture on both sides of final presentation:

```
old_mode = video.get_crt_mode()
video.set_crt_mode('curved')
sys.wait_frames(2)

rec.screenshot_composed('composed.png')
rec.screenshot_screen('screen.png')

video.set_crt_mode(old_mode)
sys.print('RESTORED ' .. video.get_crt_mode())
```

`composed.png` contains the completed composition before CRT processing, the cursor, and the status bar. `screen.png` contains the final displayed frame. The script prints the mode restored in the final two lines. Use the pair when you need to decide whether a visible difference entered during composition or during final presentation.

The CRT calls raise a script error when the selected output has no CRT controller. `video.set_crt_mode` also raises an error for any name other than `flat`, `curved`, or `off`. The two specialised screenshot calls raise an error when their capture stage is unavailable, when the path is invalid, or when capture fails or times out.

34.10 Recording Module

`rec` captures frames:

Function	Purpose
`rec.screenshot(name)`	Save one frame.
`rec.screenshot_composed(name)`	Save the composition before CRT, cursor, and status bar.
`rec.screenshot_screen(name)`	Save the final displayed frame.
`rec.start(name)`	Start recording.
`rec.start_screen(name)`	Start screen recording.
`rec.stop()`	Stop and finalise recording.
`rec.is_recording()`	Return true while recording.
`rec.frame_count()`	Number of recorded frames.

34.11 Debug Module

`dbg` drives IE Mon from a script:

Function group	Purpose
`dbg.open()`, `dbg.close()`, `dbg.is_open()`	Enter, leave, or test the monitor session.
`dbg.freeze()`, `dbg.resume()`	Aliases for `dbg.open()` and `dbg.close()`.

Function group	Purpose
`dbg.freeze_audio()`, `dbg.thaw_audio()`	Change the audio gate during the current script state.
`dbg.step()`, `dbg.continue()`, `dbg.run_until(addr)`	Execution control.
`dbg.set_bp(addr)`, `dbg.clear_bp(addr)`, `dbg.list_bp()`	Breakpoints.
`dbg.set_wp(addr)`, `dbg.clear_wp(addr)`, `dbg.list_wp()`	Watchpoints.
`dbg.get_reg(name)`, `dbg.set_reg(name, value)`	Register access.
`dbg.get_pc()`, `dbg.set_pc(addr)`	Program counter access.
`dbg.read_mem(addr, n)`, `dbg.write_mem(addr, data)`	Memory access through the monitor.
`dbg.disasm(addr, count)`	Disassemble instructions.
`dbg.backtrace()`	Return a stack backtrace.
`dbg.backtrace_frames(depth)`	Return structured backtrace frames.
`dbg.timeline(count)`	Return recent timeline entries.
`dbg.io_devices()`, `dbg.io(name)`	Inspect IE Mon I/O register views.
`dbg.history_horizon()`, `dbg.history_config(opts)`	Inspect or configure whole-machine reverse-history retention.
`dbg.tracering_on(size)`, `dbg.tracering_off()`, `dbg.tracering_show(count)`	Control the focussed CPU trace ring.
`dbg.device_list()`, `dbg.device_snapshot(name)`, `dbg.device_diff(a,b)`	Inspect versioned device snapshots.
`dbg.save_state(path)`, `dbg.load_state(path)`	Save or load a CPU-local monitor snapshot.
`dbg.on_fault(kind, fn)`	Call `fn` when a selected fault occurs.
`dbg.poll_faults()`	Poll pending fault events.
`dbg.command(line)`	Run one IE Mon command.

Fault callbacks receive a table with `cpu_id`, `pc`, `addr`, `kind`, and `info` fields.

The first `dbg.open()` or `dbg.freeze()` enters IE Mon, stops every guest CPU, and freezes the audio clock. Further nested opens only add to the script's open count. Each `dbg.close()` or `dbg.resume()` removes one open; the final matching close leaves IE Mon, resumes the CPUs that were running, and restores the audio state that existed before entry.

`dbg.freeze_audio()` and `dbg.thaw_audio()` change the audio gate directly, but the final close still performs that pre-entry restoration. When a script deliberately wants its audio choice to survive monitor exit, use `dbg.command("fa")` or `dbg.command("ta")`. Those are explicit IE Mon commands and become the new intended state.

The debug module mirrors the monitor where scripts need repeatable inspection. `dbg.io_devices()` returns the monitor's named I/O register views, and `dbg.io(name)` returns one table entry per register with `name`, `addr`, `value`, and `access` fields. The values use the same native-width MMIO read path as IE Mon `io`, so a script inspecting a long register gets a long register value even when the focussed CPU is a narrow bus client. An unknown view name returns an empty table rather than raising an error; check `dbg.io_devices()` before relying on a view name. MIDI has two useful views: `midiplay` for the file player and `midilive` for the byte stream port.

Trace-ring helpers return structured recent-instruction entries, `dbg.backtrace_frames()` returns one table per call frame, and the history helpers report or configure the whole-machine reverse timeline used by IE Mon `rg` and `rt`. Device helpers

snapshot registered versioned devices and compare two snapshots without forcing the script to parse monitor text.

`dbg.save_state` and `dbg.load_state` use IE Mon `ss` and `sl`, so they are CPU-local snapshots. They are not whole-machine save files and do not include other CPUs, device state, audio/video state, timers, DMA, or reverse-history retention.

34.12 Symbols, Regions, and Bits

Module	Useful functions
`sym`	`add`, `lookup`, `resolve`, `list`
`regions`	`list`, `lookup`
`bit32`	`band`, `bor`, `bxor`, `bnot`, `lshift`, `rshift`, `arshift`, `lrotate`, `rrotate`, `btest`, `extract`, `replace`

34.13 Runnable Video Example

Store this as `FIRST.IES`, then run `RUN "FIRST.IES"` from BASIC or `script FIRST.IES` from IE Mon:

```
-- Draw a blue VideoChip field with a green diagonal.
sys.print("IE SCRIPT VIDEO")
video.write_reg(983044, 4)
video.write_reg(983172, 1048576)
video.write_reg(983040, 1)
video.blit_fill(1048576, 320, 200, 255, 1280)
video.blit_line(0, 0, 319, 199, 65280)
video.blit_wait()
sys.print("BLT " .. video.read_reg(983108))
sys.quit()
```

The comment marks the visible job before the device writes begin. The script sets VideoChip mode `4` (`320` by `200`), selects framebuffer base `$100000`, enables the chip, fills the framebuffer, draws a diagonal, waits for the blitter, and prints the blitter status. The expected status is `BLT 2`, meaning DONE set and ERR clear. Try changing `65280` to `16711680` in the `video.blit_line` call; the diagonal changes from green to red.

34.14 Runnable Audio Example

This companion script uses the same bus-facing style for sound. It programs SoundChip channel `0` through the flexible-channel registers, waits for a short time, and prints back the control register so you can see the gate bit that was written.

Store this as `TONE.IES`:

```
-- SoundChip channel 0, square wave at about middle C.
audio.start()
audio.write_regs({
  {0xF0A80, 262 * 256},
  {0xF0A84, 96},
  {0xF0AA4, 0},
  {0xF0A88, 3}
})
sys.wait_ms(250)
sys.print("CH0 " .. mem.read32(0xF0A88))
sys.quit()
```

`audio.start()` enables the global audio path. The frequency register uses 16.8 fixed-point hertz, so `262 * 256` means about `262` Hz. `audio.write_regs` applies the four writes in their listed order. The volume write sets an audible level, `WAVE_TYPE` `0` selects a square wave, and control value `3` means enabled plus gate. The print should include `CH0 3`.

34.15 Fault Callback Example

This pattern records the program counter for any IE64 illegal instruction fault and then exits cleanly after a short wait:

```
local seen = 0

dbg.on_fault("ie64.illegal", function(ev)
  seen = ev.pc
  sys.print("FAULT PC " .. seen)
end)

sys.wait_ms(100)
sys.quit()
```

34.16 Limits and Error Behaviour

Scripts run cooperatively. A script that loops forever without calling a wait function cannot be cancelled promptly.

Storage helpers are limited to approved script storage. Names that escape that storage are rejected before any read or write occurs.

Raw RAM access through `mem` requires `cpu.freeze()`. MMIO access does not. If a script fails after freezing a CPU, audio, or the monitor, IE Script releases those holds before returning control.

34.17 What Comes Next

Part IV ends here. Part V covers persistent storage, machine control commands, input MMIO, and the serial interface.